```
var sampleLinks = [];
  var y = 0;
  while (y < 1000) {
    var randomSource = Math.floor(Math.random() * 1000);
    var randomTarget = Math.floor(linkScale(Math.random()));
    var linkObject = {source: sampleNodes[randomSource], target:
        sampleNodes[randomTarget]}
    if (randomSource != randomTarget) {
        sampleLinks.push(linkObject);
    }
    y++;
}
```

The source of each link is purely random.

The target is weighted toward popular nodes.

Don't keep any links that have the same source as target.

With this generator in place, we can instantiate our typical force-directed layout using the code in the following listing, and create a few lines and circles with it.

Listing 11.11 Force-directed layout

```
var force = d3.layout.force()
   .size([500,500])
   .gravity(.5)
   .nodes(sampleNodes)
   .links(sampleLinks)
   .on("tick", forceTick);

d3.select("svg")
   .selectAll("line.link")
   .data(sampleLinks)
   .enter()
   .append("line")
   .attr("class", "link");

d3.select("svg").selectAll("circle.node")
   .data(sampleNodes)
   .enter()
   .append("circle")
   .attr("r", 3)
   .attr("class", "node");

force.start();

function forceTick() {
  d3.selectAll("line.link")
     .attr("x1", function(d) {return d.source.x})
     .attr("y1", function(d) {return d.source.y})
     .attr("x2", function(d) {return d.target.x})
     .attr("y2", function(d) {return d.target.y});

  d3.selectAll("circle.node")
     .attr("cx", function(d) {return d.x})
     .attr("cy", function(d) {return d.y});
};
```

This is all vanilla force-directed layout code like in chapter 6.

For our initial implementation, we render everything in SVG and update the SVG on every tick.

This code should be familiar to you if you've read chapter 6. Generation of random networks is a complex and well-described practice. This random generator isn't going to win any awards, but it does produce a recognizable structure. Typical results are